

Program 5

```
#include <mpi.h>
#include <stdio.h>
#include <string.h>
int main(int argc, char** argv) {
    int rank, size;
    MPI_Status status;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if (rank == 0) {
        for (int i = 1; i < size; i++) {
            char message[100];
            sprintf(message, "Hello from process 0 to process %d", i);
            MPI_Send(message, strlen(message) + 1, MPI_CHAR, i,
0,MPI_COMM_WORLD);
            printf("Process 0 sent message to process %d\n", i);}}
        else {
            char recv_buf[100];
            MPI_Recv(recv_buf, 100, MPI_CHAR, 0, 0,
MPI_COMM_WORLD,&status);
            printf("Process %d received message: %s\n", rank, recv_buf);}
            MPI_Finalize();
            return 0; }
```

Program 6

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char** argv) {
    int rank, size;
    int send_data, recv_data;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if (rank == 0) {
        send_data = 100;
        MPI_Send(&send_data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        MPI_Recv(&recv_data, 1, MPI_INT, 1, 0,
        MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Process 0 received %d\n", recv_data);}
    else if (rank == 1) {
        send_data = 200;
        MPI_Send(&send_data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
        MPI_Recv(&recv_data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
        MPI_STATUS_IGNORE);
        printf("Process 1 received %d\n", recv_data);}
    MPI_Finalize();
    return 0; }
```

Program 7

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char** argv) {
int rank, size;
int number;
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
if (rank == 0) {
number = 42;
printf("Process 0 is broadcasting number %d\n", number); }
MPI_Bcast(&number, 1, MPI_INT, 0, MPI_COMM_WORLD);
printf("Process %d received number %d\n", rank, number);
MPI_Finalize();
return 0; }
```

Program 8

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char** argv) {
    int rank, size; int data[100];
    int recv_value;
    int gathered[100];
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if (rank == 0) {
        for (int i = 0; i < size; i++) {
            data[i] = i * 10; }
        printf("Process 0 initialized data: ");
        for (int i = 0; i < size; i++) {
            printf("%d ", data[i]); }
        printf("\n"); }
    MPI_Scatter(data, 1, MPI_INT, &recv_value, 1, MPI_INT, 0,
        MPI_COMM_WORLD);
    printf("Process %d received value %d from Scatter\n", rank,
        recv_value);
    recv_value += rank;
    MPI_Gather(&recv_value, 1, MPI_INT, gathered, 1, MPI_INT,
        0, MPI_COMM_WORLD);
    if (rank == 0) {
        printf("Process 0 gathered data: ");
        for (int i = 0; i < size; i++) {
            printf("%d ", gathered[i]); }
        printf("\n"); }
    MPI_Finalize();
    return 0; }
```

Program 9

```
#include <mpi.h> #include <stdio.h>
int main(int argc, char** argv) {
    int rank, size; int value;
    int sum_result, prod_result, max_result, min_result;
    int all_sum, all_prod, all_max, all_min;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    value = rank + 1; // Example: rank 0 = 1, rank 1 = 2, etc.
    printf("Process %d has value %d\n", rank, value);
    MPI_Reduce(&value, &sum_result, 1, MPI_INT, MPI_SUM, 0,
    MPI_COMM_WORLD);
    MPI_Reduce(&value, &prod_result, 1, MPI_INT, MPI_PROD, 0,
    MPI_COMM_WORLD);
    MPI_Reduce(&value, &max_result, 1, MPI_INT, MPI_MAX,
    0, MPI_COMM_WORLD);
    MPI_Reduce(&value, &min_result, 1, MPI_INT, MPI_MIN, 0,
    MPI_COMM_WORLD);
    if (rank == 0) {
        printf("\n--- Results using MPI_Reduce (only on root) ---\n");
        printf("Sum = %d\n", sum_result);
        printf("Prod = %d\n", prod_result);
        printf("Max = %d\n", max_result);
        printf("Min = %d\n", min_result); }
    MPI_Allreduce(&value, &all_sum, 1, MPI_INT, MPI_SUM,
    MPI_COMM_WORLD);
    MPI_Allreduce(&value, &all_prod, 1, MPI_INT, MPI_PROD,
    MPI_COMM_WORLD);
    MPI_Allreduce(&value, &all_max, 1, MPI_INT, MPI_MAX,
    MPI_COMM_WORLD);
    MPI_Allreduce(&value, &all_min, 1, MPI_INT, MPI_MIN,
    MPI_COMM_WORLD);
    printf("Process %d - AllReduce: Sum=%d Prod=%d Max=%d Min=%d\n",
    rank, all_sum, all_prod, all_max, all_min); MPI_Finalize(); return 0; }
```